

Proyecto 1

Josué Campos Herrera

Randy Fernandez Aguilar

I. INTRODUCCIÓN

La verificación funcional es una etapa fundamental en el desarrollo de circuitos digitales, ya que permite comprobar que un diseño cumple con el comportamiento esperado bajo diferentes condiciones de operación. En este proyecto se desarrolló un ambiente de verificación en SystemVerilog para el módulo `bs_gnrtr_n_rbtr`, correspondiente a un bus serial con arbitraje distribuido entre varias terminales.

El ambiente de verificación fue diseñado para generar tráfico de manera aleatoria y también para ejecutar diferentes escenarios dirigidos. A partir de las transacciones generadas, se realiza un seguimiento de los paquetes que ingresan al bus y se compara el comportamiento observado con el comportamiento esperado mediante un modelo de referencia. De esta manera, es posible detectar pérdidas, duplicaciones, entregas incorrectas o alteraciones en el contenido de los paquetes.

Además de comprobar el funcionamiento general del bus, se realizaron pruebas específicas para estudiar diferentes situaciones del protocolo, como las transferencias punto a punto, la contención entre terminales, las transmisiones broadcast y el manejo de destinos inválidos. También se incorporó una prueba orientada a la medición de la latencia de los paquetes, cuyos resultados se representan mediante histogramas para facilitar su análisis.

II. OBJETIVOS

A. Objetivo general

Desarrollar un ambiente de verificación funcional en SystemVerilog que permita comprobar el correcto funcionamiento del módulo `bs_gnrtr_n_rbtr` mediante la generación de diferentes escenarios de tráfico, la comparación de los resultados obtenidos con un modelo de referencia y el análisis de la latencia de los paquetes transmitidos.

B. Objetivos específicos

- Implementar un ambiente de verificación capaz de generar transacciones aleatorias y dirigidas hacia el módulo bajo prueba.
- Comprobar el funcionamiento del bus bajo diferentes condiciones de tráfico, incluyendo transferencias punto a punto, contención entre terminales, operaciones de *broadcast* y paquetes con destinos inválidos.
- Incorporar un mecanismo de comprobación que permita identificar pérdidas, duplicaciones, transferencias inesperadas o diferencias en el contenido de los paquetes.
- Generar histogramas de las latencias obtenidas bajo diferentes tamaños de paquete y condiciones de inyección, con

el fin de observar cómo cambia el comportamiento del bus.

III. DESCRIPCIÓN DEL MÓDULO BAJO PRUEBA

El DUT utilizado, es `bs_gnrtr_n_rbtr`. Este módulo implementa un bus serial con arbitraje distribuido, en el cual varias terminales pueden solicitar el acceso al bus y transmitir paquetes hacia otras terminales. Para este proyecto se trabajó con una configuración de cuatro terminales y un solo bus.

Cada terminal puede tener un paquete pendiente de transmisión. Cuando una terminal obtiene acceso al bus, el paquete se transmite de forma serial y posteriormente es entregado a la terminal indicada por la dirección contenida en el paquete. El diseño también contempla la posibilidad de realizar transmisiones tipo *broadcast*, así como el manejo de destinos que no corresponden a ninguna de las terminales disponibles.

A. Configuración del DUT

El módulo `bs_gnrtr_n_rbtr` utiliza varios parámetros que permiten definir las características del bus. En las pruebas realizadas se utilizaron principalmente los valores mostrados en la Tabla I.

TABLE I
PARÁMETROS PRINCIPALES DEL DUT.

Parámetro	Valor	Descripción
<code>bits</code>	1	Cantidad de buses en paralelo
<code>drvrs</code>	4	Cantidad de terminales por bus
<code>pckg_sz</code>	16	Tamaño del paquete en bits
<code>broadcast</code>	0xFF	Dirección utilizada para broadcast

El tamaño del paquete no está limitado a los 16 bits utilizados en las pruebas funcionales. El ambiente permite modificar este parámetro para realizar pruebas con diferentes tamaños. Para el análisis de latencia se utilizaron paquetes de 16, 32 y 64 bits, con el propósito de observar el efecto que tiene el tamaño del paquete sobre el tiempo necesario para completar una transferencia.

B. Interfaz del bus

La comunicación entre el DUT y el ambiente de verificación se realiza mediante las señales de la interfaz `bus_if`. Estas señales permiten indicar cuándo una terminal tiene información pendiente, proporcionar el paquete que desea transmitir y observar las operaciones realizadas por el bus.

TABLE II
SEÑALES PRINCIPALES UTILIZADAS PARA LA COMUNICACIÓN CON EL DUT.

Señal	Función
pnrng	Indica que una terminal tiene un paquete pendiente de transmisión.
D_pop	Contiene el paquete que la terminal ofrece al bus para su transmisión.
pop	Indica que el bus consumió el paquete que estaba pendiente en una terminal.
push	Indica que el bus realizó una entrega hacia una terminal.
D_push	Contiene el paquete recibido por la terminal de destino.
reset	Señal utilizada para llevar el DUT y el ambiente a su estado inicial.
clk	Señal de reloj utilizada para sincronizar la operación del sistema.

De esta forma, el ambiente de verificación puede controlar las señales de entrada del DUT y, al mismo tiempo, observar las respuestas producidas por el diseño. Esto permite registrar tanto el momento en que un paquete es consumido por el bus como el momento en que finalmente es entregado a su destino.

C. Estructura interna del DUT

El módulo superior `bs_gnrtr_n_rbtr` instancia una interfaz `bs_ntrfs_n_rbtr` para cada una de las terminales del sistema. En la configuración utilizada, esto significa que existen cuatro interfaces conectadas al mismo bus serial.

Cada interfaz contiene los elementos necesarios para controlar la lectura y escritura de los paquetes. Entre estos elementos se encuentran los bloques de control y los registros utilizados para realizar la conversión entre la representación paralela del paquete y su transmisión serial.

Una representación simplificada de esta estructura es:

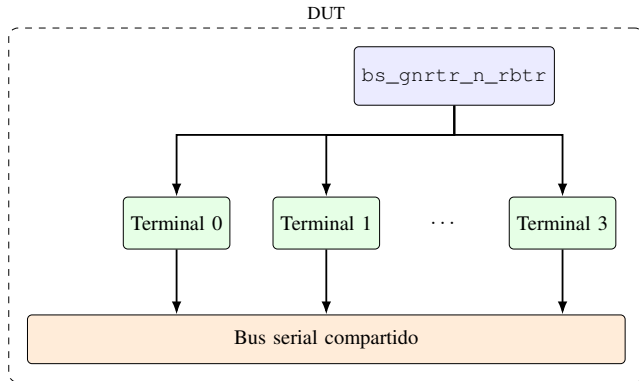


Fig. 1. Estructura simplificada del bus serial con arbitraje distribuido.

Todas las terminales comparten el bus correspondiente y utilizan las señales de control internas para determinar cuál de ellas puede realizar una transmisión. De esta manera, el acceso al medio se coordina entre las diferentes terminales sin utilizar un bloque de arbitraje central independiente.

A nivel interno, cada interfaz utiliza un bloque de serialización para convertir los datos del paquete entre su representación paralela y la forma serial utilizada por el bus. También se incluyen bloques de control encargados de

coordinar las operaciones de lectura, escritura y cambio de turno entre las terminales.

D. Funcionamiento general

Una transferencia comienza cuando una terminal indica mediante `pnrng` que posee un paquete disponible. El paquete correspondiente se presenta mediante `D_pop`. El mecanismo de arbitraje determina cuál de las terminales puede utilizar el bus y, cuando la transferencia comienza, se genera la señal `pop` correspondiente al origen.

Una vez transmitido el paquete, el sistema determina el destino a partir de la información contenida en el mismo. Cuando la transferencia corresponde a un destino válido, se genera la señal `push` en la terminal correspondiente y el paquete recibido se presenta mediante `D_push`.

En el caso de un paquete con dirección de *broadcast*, el diseño permite realizar entregas hacia las terminales correspondientes, mientras que un destino inválido no genera una entrega. Estos diferentes comportamientos fueron utilizados posteriormente como escenarios específicos dentro del plan de pruebas.

IV. ARQUITECTURA DEL AMBIENTE DE VERIFICACIÓN

Para realizar la verificación del DUT se desarrolló un ambiente basado en clases de SystemVerilog. La estructura se organizó de manera que la generación de estímulos, la aplicación de los paquetes, la observación del bus y la comprobación de los resultados fueran tareas independientes. Esto permite modificar las pruebas sin tener que cambiar todos los componentes del ambiente.

La arquitectura general está formada por los agentes, el *driver*, el monitor, el *scoreboard*, el *checker* y una clase de ambiente encargada de integrar estos componentes. La comunicación entre ellos se realiza principalmente mediante *mailboxes*, lo que permite intercambiar las transacciones sin depender directamente de la implementación interna de cada componente.

A. Estructura general

La relación entre los principales componentes del ambiente puede representarse de manera simplificada como se muestra a continuación:

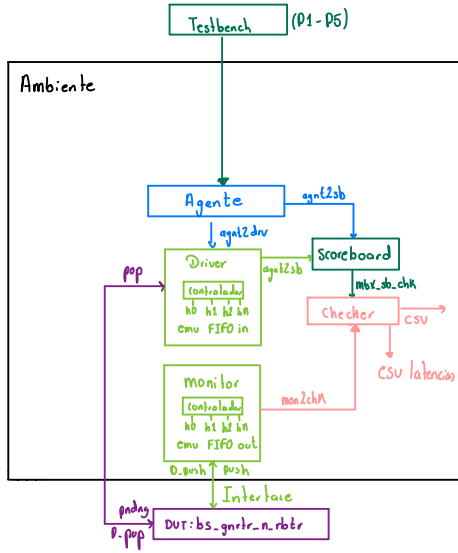


Fig. 2. Arquitectura general del ambiente de verificación.

El flujo comienza en los agentes, donde se generan las transacciones que serán utilizadas como estímulos. Estas transacciones se envían al *driver*, que las distribuye según la terminal de origen y las mantiene en una FIFO independiente para cada terminal.

El DUT recibe los paquetes a través de la interfaz *bus_if*. Mientras el diseño está funcionando, el monitor observa las señales del bus sin modificar su comportamiento. Las muestras obtenidas por el monitor son enviadas al *checker*, que se encarga de determinar si las operaciones observadas corresponden con las entregas esperadas.

De forma paralela, el *scoreboard* recibe las mismas transacciones generadas por los agentes y construye un modelo del comportamiento esperado. De esta manera, el *checker* puede comparar lo que realmente ocurrió en el DUT con lo que debía ocurrir según la transacción original.

B. Agentes

Se utiliza un agente por cada terminal del sistema. Para la configuración empleada en este proyecto se tienen cuatro agentes, correspondientes a las terminales T0, T1, T2 y T3.

Cada agente utiliza una transacción base, definida en *bus_txn*, que contiene la información necesaria para describir un paquete. La transacción se genera mediante randomización y toma en cuenta la configuración establecida para la prueba.

Entre los parámetros que pueden controlar la generación se encuentran:

- cantidad de transacciones por terminal;
- retardo mínimo y máximo entre transacciones;
- peso para destinos válidos;
- peso para mensajes de *broadcast*;
- peso para destinos inválidos;
- posibilidad de permitir envíos hacia la misma terminal de origen.

Una vez generada una transacción, el agente crea una copia para el *driver* y otra para el *scoreboard*. Esto permite que ambos componentes trabajen de manera independiente con la misma información.

El siguiente fragmento muestra la parte principal utilizada para generar y distribuir las transacciones:

```
1 repeat (n_txn > 0 ? n_txn : cfg.n_txn_per_terminal
2 ) begin
3   if (!blueprint.randomize()) begin
4     $fatal(1, "Fallo randomize()");
5   end
6   tr_drv = blueprint.copy();
7   tr_sb = blueprint.copy();
8   n_generated++;
9   agnt2sb.put(tr_sb);
10  agnt2drv.put(tr_drv);
11 end
```

De esta manera, una misma transacción sirve como referencia para el comportamiento esperado y como estímulo para el DUT.

C. Driver

El *driver* es el componente encargado de convertir las transacciones generadas por los agentes en señales que puedan ser aplicadas al DUT.

Para cada terminal se implementó una FIFO de entrada. Estas FIFO permiten almacenar los paquetes pendientes y mantenerlos disponibles hasta que el DUT indique mediante la señal *pop* que el paquete fue consumido.

El funcionamiento puede resumirse en tres pasos:

- 1) El agente genera una transacción y la envía al *driver*.
- 2) El *driver* coloca la transacción en la FIFO correspondiente a su terminal de origen.
- 3) Mientras la FIFO contenga información, el *driver* activa *pndng* y presenta el paquete mediante *D_pop*.

Cuando el DUT realiza un *pop*, el *driver* retira el primer paquete de la FIFO. De esta forma, el contenido presentado al DUT mantiene el orden en que fueron generadas las transacciones.

Además, el *driver* registra el tiempo en que cada paquete entra realmente a la FIFO de entrada. Esta información posteriormente es utilizada para calcular la latencia total de cada transferencia.

D. Monitor

El monitor funciona de manera pasiva, por lo que no modifica ninguna señal del DUT. Su función es observar el comportamiento del bus durante cada ciclo de reloj.

Para cada ciclo se crea una muestra que contiene el estado de las señales principales:

- *pndng*;
- *pop*;
- *push*;
- *D_pop*;
- *D_push*;
- estado del *reset*.

Las muestras se envían posteriormente al *checker*. Además, el monitor mantiene contadores independientes de los pop y push observados en cada terminal, lo que permite generar un resumen de las operaciones realizadas durante la simulación.

Una característica importante es que el monitor utiliza un *clocking block* de la interfaz para realizar las observaciones de forma sincronizada con el reloj. Esto permite separar la observación de las señales de la aplicación de estímulos y evita depender directamente del cambio instantáneo de las señales en el flanco del reloj.

E. Scoreboard

El *scoreboard* representa el comportamiento esperado del sistema. Para esto mantiene una FIFO esperada por cada terminal de origen y utiliza las transacciones recibidas directamente desde los agentes.

Cuando una transacción es recibida, se almacena en la FIFO correspondiente a su origen. Posteriormente, cuando el *driver* informa que el DUT consumió un paquete mediante un pop, el *scoreboard* toma la primera transacción de la FIFO correspondiente y determina qué entrega debería producirse.

El modelo contempla los diferentes tipos de destino utilizados en las pruebas:

- Para un destino válido, se genera una entrega esperada hacia la terminal indicada.
- Para un mensaje de *broadcast*, se generan las entregas correspondientes a las terminales de destino.
- Para un destino inválido, no se genera ninguna entrega esperada.

El *scoreboard* también mantiene información relacionada con los tiempos de envío. Esto permite asociar cada paquete con el instante en que fue introducido al sistema y utilizar posteriormente esta información para el cálculo de latencia.

F. Checker

El *checker* es el componente encargado de comparar las operaciones observadas por el monitor con las entregas generadas por el *scoreboard*.

Cuando el monitor detecta un push, el *checker* busca en el modelo esperado una entrega que coincida con el destino y con el contenido del paquete. Si la entrega existe, se considera correcta y se retira del modelo esperado.

También se comprueban los pop observados, verificando que exista información correspondiente al paquete que debía ser consumido.

Entre los casos que pueden ser detectados por el *checker* se encuentran:

- pop sin un paquete esperado;
- diferencias entre el paquete esperado y el observado;
- push inesperado;
- ausencia del tiempo de envío de una transacción;
- elementos que permanecen pendientes al finalizar la prueba.

Además de realizar estas comprobaciones, el *checker* calcula la latencia de cada paquete. Para ello utiliza el tiempo de

entrada registrado por el *scoreboard* y el instante en que se observa el push correspondiente.

Los resultados pueden almacenarse en un archivo CSV con información como el identificador de la transacción, origen, destino, paquete, tiempo de envío, tiempo de recepción, latencia y cantidad de ciclos. Estos archivos son utilizados posteriormente para generar los histogramas de latencia.

G. Clase Ambiente

Finalmente, todos los componentes anteriores son integrados mediante la clase *Ambiente*. Esta clase se encarga de crear los *mailboxes*, construir los diferentes componentes y establecer las conexiones entre ellos.

Las principales conexiones utilizadas son:

```
Agente -> Driver
Agente -> Scoreboard
Driver -> Scoreboard
Monitor -> Checker
Checker -> Scoreboard
```

La clase *Ambiente* también controla el inicio de los componentes, el proceso de *reset* y la espera necesaria para que el bus termine de procesar los paquetes pendientes antes de finalizar una prueba.

V. IMPLEMENTACIÓN DEL AMBIENTE DE VERIFICACIÓN

La implementación del ambiente se realizó utilizando SystemVerilog y se organizó en diferentes archivos, de manera que cada componente tuviera una responsabilidad específica. Esta separación facilita la reutilización del ambiente entre las diferentes pruebas y permite modificar la generación de estímulos sin tener que cambiar la lógica encargada de comprobar los resultados.

Los principales archivos utilizados para construir el ambiente fueron la interfaz del bus, la configuración, las transacciones, el agente, el driver, el monitor, el scoreboard, el checker y el ambiente. Sobre esta estructura se construyeron las diferentes pruebas dirigidas y la prueba de latencia.

A. Interfaz y configuración

La comunicación entre el ambiente de verificación y el DUT se realiza mediante la interfaz *bus_if*. Esta interfaz concentra las señales utilizadas por el bus y permite separar la forma en que cada componente accede a ellas.

Para evitar que todos los componentes tengan acceso de la misma manera a las señales, se utilizaron diferentes *modports*. Entre ellos se encuentran los destinados al driver, al monitor y al propio DUT.

La configuración general del ambiente se concentra en el archivo *bus_config.svh*. En este archivo se definen parámetros como la cantidad de terminales, el tamaño de los paquetes y los pesos utilizados durante la generación aleatoria.

Entre los parámetros utilizados durante las pruebas se encuentran:

- *DRVRS*: cantidad de terminales del bus.
- *PCKG_SZ*: tamaño del paquete.

- MIN_DELAY y MAX_DELAY: rango de retardo utilizado entre transacciones.
- Peso para destinos válidos.
- Peso para mensajes de *broadcast*.
- Peso para destinos inválidos.
- Permiso para realizar envíos hacia la misma terminal de origen.

La utilización de una configuración centralizada permite utilizar el mismo ambiente para diferentes escenarios. Por ejemplo, las pruebas de latencia se ejecutaron con diferentes tamaños de paquete y rangos de retardo sin modificar la estructura principal del ambiente.

B. Transacciones

La información de cada paquete se representa mediante la clase `bus_txn`. Esta clase contiene los datos necesarios para identificar el origen, destino, tipo de operación, contenido del paquete y retardo utilizado antes de su inyección al bus.

La generación de las transacciones se realiza mediante randomización. De esta forma, el agente puede producir diferentes combinaciones de paquetes a partir de una misma estructura.

Una parte importante de la implementación consiste en utilizar una misma transacción como referencia tanto para el estímulo como para el comportamiento esperado. Esto permite que el scoreboard conozca exactamente qué paquete fue generado y qué resultado debe esperar.

El flujo utilizado por el agente puede resumirse mediante el siguiente fragmento:

Listing 1. Generación y distribución de transacciones.

```
1 repeat (n_txn > 0 ? n_txn : cfg.n_txn_per_terminal
2 ) begin
3   if (!blueprint.randomize()) begin
4     $fatal(1, "Fallo randomize()");
5   end
6   tr_drv = blueprint.copy();
7   tr_sb = blueprint.copy();
8   n_generated++;
9   agnt2sb.put(tr_sb);
10  agnt2drv.put(tr_drv);
11 end
```

Después de realizar la randomización, se crean dos copias de la transacción. Una se envía al driver para ser aplicada al DUT y la otra se envía al scoreboard para construir el resultado esperado.

Esto permite que ambos componentes trabajen de manera independiente y evita que la comprobación dependa directamente de la transacción que se está aplicando al DUT.

C. Agente

El agente es el encargado de generar el tráfico utilizado durante la simulación. En el ambiente se utiliza un agente asociado a cada terminal del bus.

La generación puede realizarse de forma completamente aleatoria o siguiendo condiciones específicas establecidas por una prueba. Por ejemplo, en la prueba TP-01 se restringe el origen a T0 y el destino a T2. En cambio, en TP-04 se generan únicamente paquetes de tipo *broadcast*.

Esta posibilidad de modificar las restricciones sin cambiar el funcionamiento del resto del ambiente permite utilizar el mismo agente para diferentes escenarios.

Durante las pruebas también se registra información de las transacciones generadas. Un ejemplo de la información mostrada durante una simulación es:

```
AGNT0 id=1 src=0 dst=2 type=DST_VALID
payload=0x80 packet=0x280 delay=4
```

Este tipo de salida facilita revisar qué estímulos fueron realmente introducidos al sistema y permite relacionarlos posteriormente con los resultados obtenidos por el checker.

D. Driver

El driver recibe las transacciones generadas por los agentes y las convierte en señales que pueden ser aplicadas al DUT.

Para cada terminal se utiliza una FIFO independiente. Cuando una transacción llega al driver, se almacena en la FIFO correspondiente a su origen. Mientras existan paquetes pendientes, el driver mantiene activada la señal correspondiente a la existencia de tráfico y presenta el paquete al DUT.

Cuando el DUT indica que consumió un paquete mediante la señal `pop`, el driver retira el elemento correspondiente de la FIFO.

Esta implementación permite mantener el orden de los paquetes dentro de cada terminal y facilita el manejo de situaciones en las que varias terminales tienen paquetes pendientes al mismo tiempo.

Además, el driver mantiene información estadística sobre las operaciones realizadas. En los resultados obtenidos se reportan, entre otros datos:

- cantidad de paquetes enviados;
- ciclos durante los cuales una FIFO permaneció llena;
- intentos de realizar un `pop` sin disponer de un paquete;
- accesos fuera de rango.

Estos datos permiten detectar problemas en la aplicación de estímulos antes de analizar el comportamiento del DUT.

E. Monitor

El monitor se implementó como un componente pasivo. Su función es observar las señales del bus durante la simulación sin modificar su funcionamiento.

En cada ciclo se recopila información relacionada con las operaciones realizadas por el DUT, principalmente las señales `pop`, `push`, `D_pop` y `D_push`.

A partir de estas observaciones se mantienen contadores independientes para cada terminal. Por ejemplo, al finalizar una prueba pueden obtenerse resultados como:

```
T0: pop=20 push=0
T1: pop=0 push=0
T2: pop=0 push=20
T3: pop=0 push=0
```

Esta información permite comprobar rápidamente si las operaciones ocurrieron en las terminales esperadas.

El monitor también proporciona al checker las muestras necesarias para realizar la comparación entre el comportamiento observado y el comportamiento esperado.

F. Scoreboard

El scoreboard funciona como un modelo de referencia del sistema. Su objetivo no es reproducir internamente toda la implementación del DUT, sino determinar qué entregas deberían producirse a partir de las transacciones generadas.

Para esto mantiene información de las transacciones recibidas y de los paquetes que todavía deben ser consumidos. Cuando se observa un `pop`, el scoreboard determina el resultado esperado de acuerdo con el origen, destino y tipo de paquete.

Para un destino válido se crea una entrega esperada hacia la terminal indicada. Para un mensaje de *broadcast*, se crean las entregas correspondientes a las terminales de destino. En el caso de un destino inválido, no se crea ninguna entrega.

Esta lógica permite que el mismo modelo sea utilizado por las diferentes pruebas. Por ejemplo, en la prueba TP-04 se generaron cinco paquetes de *broadcast*. Debido a la configuración de cuatro terminales y a que el envío hacia la propia terminal estaba deshabilitado, cada paquete produjo tres entregas esperadas, para un total de quince.

El reporte obtenido durante esta prueba confirmó este comportamiento:

```
Recibidas del Agent      : 5
Consumidas por pop      : 5
Entregas creadas        : 15
Entregas retiradas      : 15
Broadcast               : 5
Entregas pendientes     : 0
```

De esta forma, el scoreboard permite verificar que el número y tipo de entregas producidas por el DUT corresponden con el comportamiento esperado.

G. Checker

El checker recibe las muestras generadas por el monitor y las compara con la información proporcionada por el scoreboard.

La comparación considera tanto las operaciones de entrada como las entregas producidas por el DUT. Entre las situaciones que se verifican se encuentran los `pop` sin información esperada, paquetes incorrectos, `push` inesperados, ausencia de información de tiempo y elementos que permanezcan pendientes al finalizar la prueba.

El resultado de estas comprobaciones se resume en un reporte. Por ejemplo, para la prueba TP-01 se obtuvo:

```
POP observados          : 20
PUSH observados         : 20
Entregas correctas      : 20
POP sin dato esperado   : 0
D_pop incorrectos       : 0
PUSH inesperados        : 0
Sin tiempo de envio     : 0
ERRORES                 : 0
```

Además de comprobar el contenido de las transferencias, el checker permite calcular la latencia de cada paquete. Para esto se utiliza el tiempo en que el paquete fue registrado como enviado y el tiempo en que se observó la entrega correspondiente.

Por ejemplo, durante TP-01 se obtuvo para el primer paquete:

```
[CHK] OK push: src=0 dst=2 packet=0x280
retraso=220 ns ciclos=22 latencia_pop=19 ciclos
```

Los resultados de latencia se almacenan en archivos CSV durante las pruebas destinadas a realizar este análisis. Posteriormente, estos archivos se utilizan para generar los histogramas.

H. Ambiente

Todos los componentes anteriores son integrados mediante la clase *Ambiente*. Esta clase se encarga de crear y conectar los diferentes componentes antes de iniciar una simulación.

Las conexiones principales pueden resumirse de la siguiente manera:

```
Agente -> Driver
Agente -> Scoreboard
Driver -> Scoreboard
Monitor -> Checker
Checker -> Scoreboard
```

El ambiente también coordina el proceso de inicialización, el reset y la espera necesaria para que el bus termine de procesar los paquetes pendientes.

Esta última parte es importante debido a que la generación de transacciones puede finalizar antes de que el DUT haya terminado de procesarlas. Por esta razón, las pruebas no terminan inmediatamente después de generar el último paquete, sino que esperan a que el bus se encuentre drenado.

En las pruebas realizadas esta condición se reporta mediante mensajes como:

```
[ENV] bus drenado tras 502 ciclos
```

Una vez que no existen elementos pendientes, se generan los reportes finales del driver, monitor, scoreboard y checker, y posteriormente la prueba determina su resultado.

I. Organización de las pruebas

Sobre el ambiente común se desarrollaron las diferentes pruebas del proyecto. Las pruebas dirigidas modifican principalmente la configuración y las restricciones de generación, mientras que los componentes principales permanecen sin cambios.

La estructura utilizada permite distinguir entre la infraestructura de verificación y el escenario que se desea comprobar. De esta manera, una prueba puede concentrarse en una característica particular del DUT sin tener que implementar nuevamente el driver, monitor o scoreboard.

Las pruebas implementadas fueron:

- `TestGeneral`: tráfico aleatorio general.

- **TestTP1:** transferencia punto a punto entre dos terminales.
- **TestTP2:** ejercicio de las diferentes rutas de comunicación.
- **TestTP3:** prueba de contención entre terminales.
- **TestTP4:** funcionamiento del *broadcast*.
- **TestTP5:** comportamiento ante destinos inválidos.
- **TestLatencia:** medición de latencia y generación de archivos CSV.

En este proyecto la verificación se basa en la comparación realizada mediante el scoreboard y el checker, además de los contadores y métricas obtenidos durante las simulaciones. No se utilizaron mecanismos adicionales de *coverage* ni assertions como parte del ambiente final.

J. Flujo completo de una transacción

Considerando todos los componentes anteriores, el recorrido de una transacción dentro del ambiente puede resumirse de la siguiente forma:

- 1) El agente genera una transacción de acuerdo con la configuración de la prueba.
- 2) Se crean copias de la transacción para el driver y el scoreboard.
- 3) El driver almacena el paquete en la FIFO correspondiente a su terminal.
- 4) El paquete es presentado al DUT mediante la interfaz del bus.
- 5) El DUT consume el paquete mediante una operación *pop*.
- 6) El scoreboard determina qué resultado se espera para esa transacción.
- 7) El monitor observa las operaciones realizadas por el DUT.
- 8) Cuando se produce un *push*, el checker compara el resultado observado con el resultado esperado.
- 9) Si la transferencia es correcta, se registra como una entrega confirmada.
- 10) Finalmente, se comprueba que no existan elementos pendientes y se genera el reporte de la prueba.

Este flujo permite verificar el comportamiento del bus desde la generación del estímulo hasta la entrega final del paquete, manteniendo separadas las funciones de generación, aplicación, observación y comprobación.

VI. PLAN DE PRUEBAS

El plan de pruebas se diseñó con el objetivo de comprobar el funcionamiento del bus desde diferentes situaciones de operación. Se utilizaron pruebas generales, pruebas dirigidas y una prueba específica para medir la latencia. Cada escenario busca comprobar una característica particular del sistema y, en conjunto, permiten verificar tanto las transferencias normales como algunos casos especiales del protocolo.

Las pruebas se ejecutaron utilizando el mismo ambiente de verificación. La diferencia entre los escenarios se establece principalmente mediante la configuración de los agentes y las restricciones aplicadas a las transacciones. De esta manera, se

mantiene una estructura común y se modifica únicamente el tipo de tráfico que se desea evaluar.

A. Criterios generales de verificación

Para determinar el resultado de cada prueba se utilizaron seis criterios principales. Estos criterios permiten revisar tanto la generación de estímulos como el comportamiento observado en el bus y el estado final del ambiente.

- **C1 – Generación de estímulos:** comprobar que los paquetes sean generados de acuerdo con las condiciones establecidas para la prueba.
- **C2 – Operaciones de POP:** verificar que los paquetes sean consumidos por la terminal de origen correspondiente.
- **C3 – Operaciones de PUSH:** comprobar que las entregas ocurran en las terminales que corresponden según el tipo de paquete.
- **C4 – Entregas esperadas:** verificar que el scoreboard genere y retire correctamente las entregas que deberían producirse.
- **C5 – Contenido y protocolo:** comprobar que el contenido observado corresponda con el paquete esperado y que las operaciones del protocolo sean correctas.
- **C6 – Estado final:** verificar que no queden paquetes o entregas pendientes al finalizar la simulación.

Además de estos criterios, el checker mantiene un contador de errores y reporta situaciones como POP sin información esperada, *D_pop* incorrectos, PUSH inesperados y ausencia de información de tiempo.

B. Prueba general

La prueba *TestGeneral* utiliza tráfico aleatorio para comprobar el comportamiento general del bus. Los agentes generan paquetes con diferentes orígenes, destinos y retardos, utilizando los pesos definidos en la configuración.

El propósito de esta prueba es observar el comportamiento del sistema bajo una combinación de situaciones sin restringir el tráfico a una única ruta. Esto permite utilizar el ambiente de una manera similar a las condiciones que podrían presentarse durante una operación normal del bus.

TABLE III
PLAN DE PRUEBA PARA TESTGENERAL.

Elemento	Descripción
Escenario	Tráfico aleatorio entre las terminales del bus.
Estímulo	Transacciones generadas aleatoriamente según la configuración.
Objetivo	Comprobar el funcionamiento general del ambiente y del DUT.
Verificación	Comparación entre las operaciones observadas y el modelo de referencia.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	No deben existir pérdidas, duplicaciones, entregas incorrectas ni elementos pendientes.

C. Prueba TP-01: transferencia punto a punto

La prueba *TestTP1* comprueba una transferencia directa entre dos terminales específicas. En el escenario utilizado, la terminal T0 actúa como origen y la terminal T2 como destino.

Durante la prueba se generan 20 paquetes desde T0 con destino T2. Los paquetes utilizan destinos válidos y no se habilitan mensajes de *broadcast* ni destinos inválidos.

El objetivo principal es comprobar que todos los paquetes enviados desde la terminal de origen sean consumidos correctamente y posteriormente entregados únicamente a la terminal de destino.

TABLE IV
PLAN DE PRUEBA TP-01.

Elemento	Descripción
Origen	T0.
Destino	T2.
Cantidad	20 paquetes.
Tipo de destino	Destino válido.
Broadcast	Deshabilitado.
Destino inválido	Deshabilitado.
Objetivo	Verificar una transferencia punto a punto.
Verificación	POP en T0 y PUSH en T2.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	Los 20 paquetes deben llegar a T2 sin errores ni elementos pendientes.

D. Prueba TP-02: rutas origen-destino

La prueba TestTP2 tiene como propósito ejercitar las diferentes rutas posibles entre las terminales del bus. Para un sistema con cuatro terminales se consideran las combinaciones entre diferentes terminales de origen y destino, evitando los envíos hacia la misma terminal cuando esta condición se encuentra deshabilitada.

Esta prueba permite verificar que el comportamiento observado no dependa únicamente de una ruta específica y que las diferentes combinaciones de origen y destino sean procesadas correctamente.

TABLE V
PLAN DE PRUEBA TP-02.

Elemento	Descripción
Escenario	Diferentes combinaciones origen-destino.
Terminales	4.
Tipo de tráfico	Destinos válidos.
Self-send	Deshabilitado.
Objetivo	Ejercitar las diferentes rutas de comunicación.
Verificación	Comparar cada entrega observada con su origen y destino esperado.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	Cada paquete debe ser entregado en la terminal correspondiente a su destino.

E. Prueba TP-03: contención entre terminales

La prueba TestTP3 está orientada a comprobar el comportamiento del arbitraje cuando varias terminales tienen paquetes pendientes al mismo tiempo.

Para aumentar la contención, las terminales generan tráfico con un retardo mínimo, de manera que varias solicitudes pueden encontrarse pendientes simultáneamente. El objetivo es verificar que el bus pueda atender las diferentes terminales sin perder paquetes y que las entregas mantengan la correspondencia con las transacciones generadas.

TABLE VI
PLAN DE PRUEBA TP-03.

Elemento	Descripción
Escenario	Varias terminales transmitiendo simultáneamente.
Retardo	Se utiliza un retardo mínimo para aumentar la contención.
Origen	Varias terminales.
Objetivo	Comprobar el funcionamiento del arbitraje distribuido.
Verificación	Revisar que los paquetes de las diferentes terminales sean procesados correctamente.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	Todos los paquetes válidos deben ser procesados y entregados sin pérdida ni duplicación.

F. Prueba TP-04: broadcast

La prueba TestTP4 comprueba el funcionamiento de los paquetes de tipo *broadcast*. En este escenario, los paquetes son enviados desde T0 utilizando la dirección de difusión.

Para la configuración utilizada, el bus cuenta con cuatro terminales y el envío hacia la propia terminal de origen se encuentra deshabilitado. Por esta razón, cada paquete de *broadcast* debe producir tres entregas, correspondientes a T1, T2 y T3.

La prueba utilizada genera cinco paquetes de *broadcast*, por lo que se esperan quince entregas en total.

TABLE VII
PLAN DE PRUEBA TP-04.

Elemento	Descripción
Origen	T0.
Dirección	0xFF.
Cantidad	5 paquetes.
Terminales receptoras	T1, T2 y T3.
Broadcast hacia origen	Deshabilitado.
Entregas esperadas	15.
Objetivo	Verificar la distribución de un paquete a todos los destinos correspondientes.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	Cada paquete debe producir tres entregas, para un total de quince.

G. Prueba TP-05: destino inválido

La prueba TestTP5 permite comprobar el comportamiento del bus cuando recibe un paquete cuyo destino no corresponde con ninguna de las terminales disponibles.

En el escenario utilizado se generan cinco paquetes desde T0 utilizando como destino la dirección 0x20. Debido a que el sistema cuenta con cuatro terminales, esta dirección no corresponde a una terminal válida.

En este caso se espera que los paquetes sean consumidos por el bus, pero que no se produzca ninguna operación de PUSH. El scoreboard debe reconocer estos paquetes como destinos inválidos y no generar entregas esperadas.

TABLE VIII
PLAN DE PRUEBA TP-05.

Elemento	Descripción
Origen	T0.
Destino	0x20.
Cantidad	5 paquetes.
Tipo de destino	Inválido.
Objetivo	Verificar el descarte de paquetes con destinos no válidos.
Verificación	POP en T0 y ausencia de PUSH.
Criterios	C1, C2, C3, C4, C5 y C6.
Resultado esperado	Los cinco paquetes deben ser consumidos sin generar entregas.

H. Prueba de latencia

Además de las pruebas funcionales, se desarrolló `TestLatencia` para obtener información sobre el tiempo que tarda un paquete en recorrer el sistema.

En esta prueba se registran los tiempos asociados con cada paquete y se calcula la latencia correspondiente. Los resultados se almacenan en un archivo CSV, que posteriormente puede utilizarse para generar histogramas mediante `GNUplot`.

La prueba puede ejecutarse modificando parámetros como el tamaño del paquete, el número de transacciones y el rango de retardo. Esto permite observar cómo cambia la distribución de las latencias bajo diferentes condiciones de operación.

Un ejemplo de ejecución es:

Listing 2. Ejecución de una prueba de latencia.

```
1 make run TOP=TestLatencia \
2   PKG_SZ=16 MIN_DELAY=0 MAX_DELAY=5 \
3   N_TXN=100 SEED=20 \
4   MAX_CYCLES=40000 DRAIN_CYCLES=20000 \
5   CSV=Reportes/latencias_16b_delay0_5.csv
6
7 make plot CSV=Reportes/latencias_16b_delay0_5.csv
```

Para realizar las diferentes corridas se modifican principalmente `PKG_SZ`, `MIN_DELAY`, `MAX_DELAY`, `N_TXN` y la semilla utilizada. Los parámetros `MAX_CYCLES` y `DRAIN_CYCLES` deben ajustarse de acuerdo con el tamaño del paquete y la cantidad de tráfico, ya que el bus es serial y requiere más ciclos para procesar paquetes de mayor tamaño.

I. Resumen del plan de pruebas

La Tabla IX resume los escenarios considerados en el proyecto.

En conjunto, estas pruebas permiten evaluar el comportamiento funcional del bus en condiciones normales y en situaciones específicas que pueden afectar su funcionamiento. La información obtenida de cada prueba se utiliza posteriormente para analizar los resultados de la verificación y determinar si se cumplieron los objetivos establecidos.

VII. RESULTADOS

En esta sección se presentan los resultados obtenidos durante la verificación funcional del módulo `bs_gnrtr_n_rbtr`. Se incluyen tanto las pruebas funcionales dirigidas como las mediciones de latencia

obtenidas bajo diferentes tamaños de paquete y rangos de retardo.

Las pruebas funcionales permitieron comprobar el comportamiento del bus ante transferencias punto a punto, operaciones de broadcast y destinos inválidos. Por otra parte, las pruebas de latencia permitieron observar cómo cambia el retraso de los paquetes al modificar el tamaño del paquete y el intervalo utilizado para la inyección de tráfico.

A. Resultados de las pruebas funcionales

Las pruebas funcionales fueron ejecutadas utilizando el ambiente de verificación desarrollado en `SystemVerilog`. En todos los casos se compararon las transferencias observadas en el bus con el comportamiento esperado generado por el modelo de referencia.

1) *Transferencia punto a punto:* La prueba TP-01 verificó una transferencia directa desde la terminal T0 hacia la terminal T2. Se generaron 20 paquetes con un tamaño de 16 bits.

Los resultados obtenidos muestran que los 20 paquetes fueron consumidos por T0 y posteriormente entregados correctamente a T2. No se presentaron paquetes pendientes, transferencias inesperadas ni errores en el contenido de los paquetes.

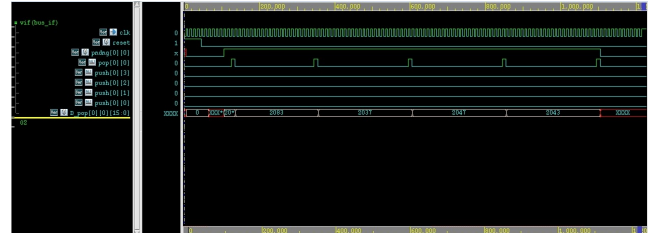


Fig. 3. Forma de onda correspondiente a la transferencia punto a punto de TP-01.

La forma de onda permite observar la actividad del reloj, las señales de solicitud de las terminales y las transferencias realizadas por el bus. También se puede observar la transmisión de los datos asociados a los paquetes generados.

En el reporte de la prueba se obtuvieron los siguientes resultados:

- 20 paquetes generados.
- 20 operaciones de POP.
- 20 operaciones de PUSH.
- 20 entregas confirmadas por el scoreboard.
- 0 elementos pendientes al finalizar la simulación.
- 0 errores reportados por el checker.

Por lo tanto, la prueba TP-01 finalizó con resultado **PASS**.

2) *Verificación de broadcast:* La prueba TP-04 se utilizó para verificar el comportamiento de los paquetes enviados mediante la dirección de broadcast. En esta prueba se generaron cinco paquetes desde T0 utilizando la dirección 0xFF.

Debido a que el broadcast no entrega el paquete nuevamente a su terminal de origen, cada paquete generó tres entregas, correspondientes a T1, T2 y T3. En total se esperaban 15 entregas.

TABLE IX
RESUMEN DEL PLAN DE PRUEBAS.

Prueba	Escenario	Característica principal
TestGeneral	Tráfico aleatorio	Funcionamiento general del bus.
TP-01	T0 → T2	Transferencia punto a punto.
TP-02	Diferentes rutas	Comunicación entre diferentes terminales.
TP-03	Varias terminales transmitiendo	Contención y arbitraje.
TP-04	T0 → T1,T2,T3	Funcionamiento de <i>broadcast</i> .
TP-05	Destino 0x20	Manejo de destinos inválidos.
TestLatencia	Diferentes tamaños y retardos	Medición y distribución de latencia.

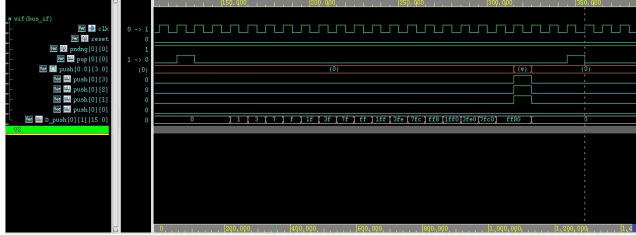


Fig. 4. Forma de onda correspondiente a la operación de broadcast.

Los resultados de esta prueba fueron:

- 5 paquetes broadcast generados.
- 5 operaciones de POP en T0.
- 15 operaciones de PUSH.
- 15 entregas confirmadas por el scoreboard.
- 0 entregas pendientes.
- 0 errores reportados por el checker.

La prueba TP-04 finalizó con resultado **PASS**, confirmando que el mecanismo de broadcast realizó las entregas correspondientes a las terminales de destino.

3) *Destino inválido*: La prueba TP-05 permitió comprobar el comportamiento del bus cuando se utiliza una dirección de destino que no corresponde a ninguna de las terminales disponibles. Para esta prueba se utilizaron cinco paquetes con destino 0x20.

Los paquetes fueron consumidos por la terminal de origen, pero no se generaron entregas hacia ninguna terminal. El scoreboard identificó correctamente los cinco paquetes como destinos inválidos.

- 5 paquetes con destino inválido.
- 5 operaciones de POP.
- 0 operaciones de PUSH.
- 5 destinos inválidos detectados por el scoreboard.
- 0 elementos pendientes.
- 0 errores reportados por el checker.

La prueba TP-05 finalizó con resultado **PASS**. Esto confirma que el ambiente de verificación detecta correctamente los destinos inválidos y que el DUT no genera entregas para ellos.

B. Formas de onda adicionales

Además de las formas de onda utilizadas para las pruebas específicas, se analizaron diferentes instantes de la simulación para observar con mayor detalle el funcionamiento del arbitraje y la transferencia de los datos.



Fig. 5. Actividad del arbitraje entre las terminales del bus.

La Figura 5 muestra diferentes operaciones de POP realizadas sobre las terminales durante la simulación. Estas señales permiten observar que el bus atiende las solicitudes pendientes de las distintas terminales.

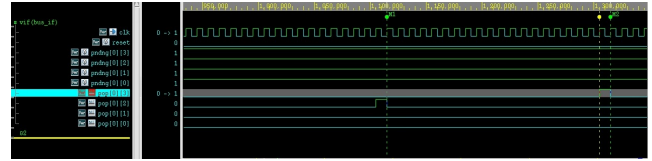


Fig. 6. Detalle de una operación de arbitraje durante la simulación.

En esta vista se puede apreciar con mayor detalle el instante en que una terminal es seleccionada para realizar una transferencia y cómo las señales del bus cambian durante dicha operación.

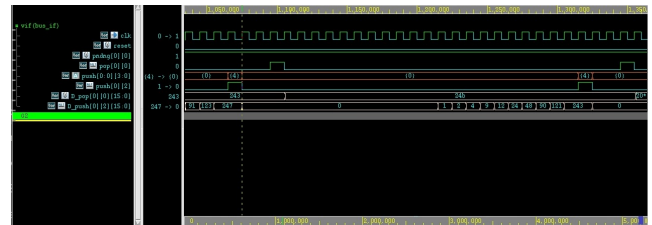


Fig. 7. Detalle de la transferencia de datos entre terminales.

La Figura 7 permite observar la relación entre las señales de solicitud, la operación de POP y los datos transmitidos. Estas formas de onda sirvieron como apoyo para comprobar visualmente el comportamiento observado en los reportes de las pruebas.

C. Resultados de latencia

Además de verificar el funcionamiento lógico del bus, se realizaron pruebas para medir la latencia de los paquetes. Para estas pruebas se utilizaron 400 paquetes y se modificaron dos parámetros:

- 1) El tamaño del paquete, utilizando 16, 32 y 64 bits.
- 2) El intervalo de retardo utilizado entre las inyecciones de paquetes.

La latencia registrada corresponde al número de ciclos de reloj transcurridos desde la inyección del paquete hasta su entrega. Los resultados se representan mediante histogramas, donde el eje horizontal corresponde al retraso total y el eje vertical representa la cantidad de paquetes dentro de cada intervalo.

1) *Paquetes de 16 bits*: Para paquetes de 16 bits se realizaron tres configuraciones diferentes de retardo.

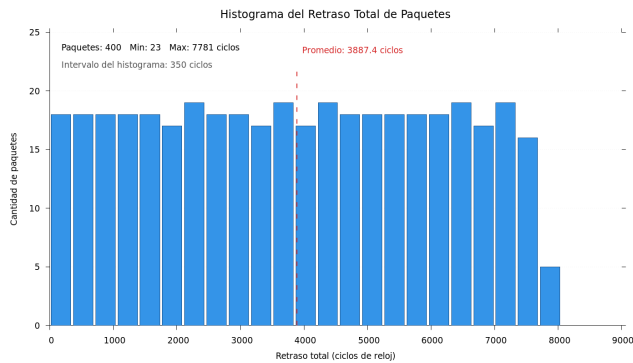


Fig. 8. Latencia para paquetes de 16 bits con retardo de 0–5 ciclos.

Para esta configuración se obtuvo un promedio de 3887.4 ciclos, con un mínimo de 23 ciclos y un máximo de 7781 ciclos.

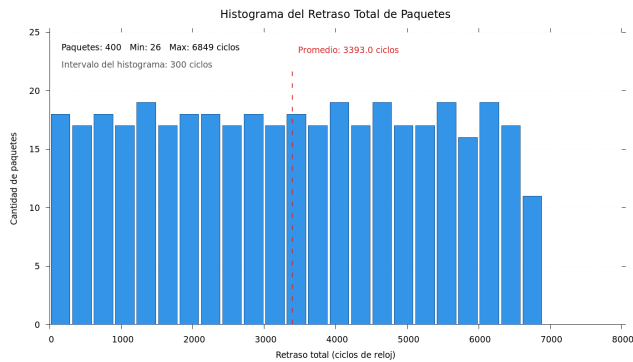


Fig. 9. Latencia para paquetes de 16 bits con retardo de 5–20 ciclos.

En esta prueba el promedio fue de 3393.0 ciclos, con un mínimo de 26 ciclos y un máximo de 6849 ciclos.

Para el intervalo de 20–100 ciclos se obtuvo un promedio de 1043.0 ciclos, con un mínimo de 22 ciclos y un máximo de 2467 ciclos.

2) *Paquetes de 32 bits*: También se realizaron pruebas utilizando paquetes de 32 bits.

El promedio obtenido fue de 7095.4 ciclos, con un mínimo de 39 ciclos y un máximo de 14181 ciclos.

En esta configuración se obtuvo un promedio de 6601.0 ciclos, con un mínimo de 42 ciclos y un máximo de 13249 ciclos.

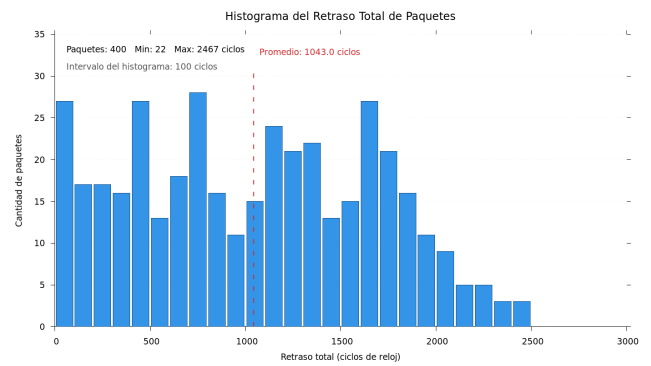


Fig. 10. Latencia para paquetes de 16 bits con retardo de 20–100 ciclos.

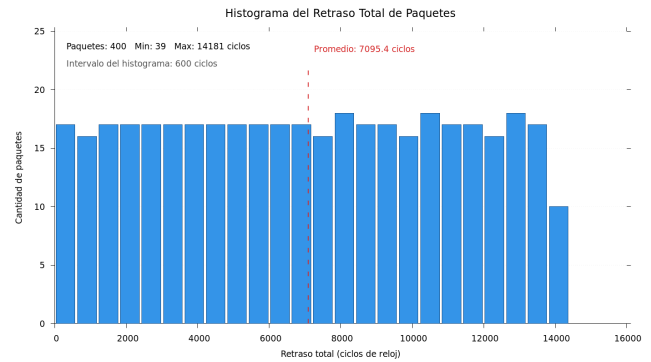


Fig. 11. Latencia para paquetes de 32 bits con retardo de 0–5 ciclos.

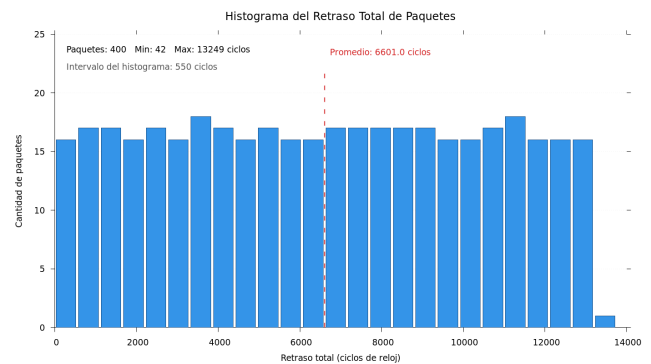


Fig. 12. Latencia para paquetes de 32 bits con retardo de 5–20 ciclos.

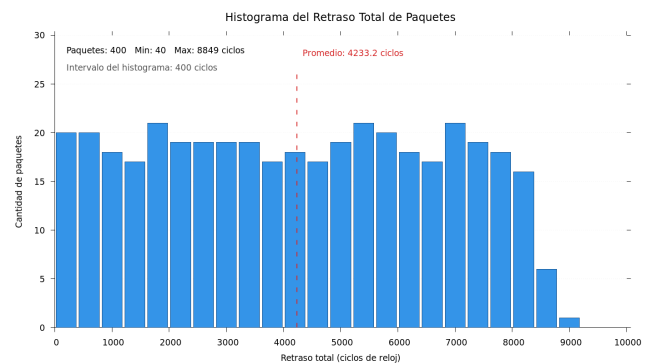


Fig. 13. Latencia para paquetes de 32 bits con retardo de 20–100 ciclos.

Para este caso el promedio registrado fue de 4233.2 ciclos, con un mínimo de 40 ciclos y un máximo de 8849 ciclos.

3) *Paquetes de 64 bits*: Finalmente, se realizaron pruebas utilizando paquetes de 64 bits.

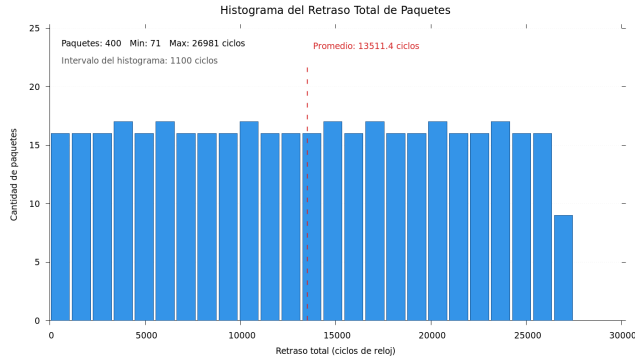


Fig. 14. Latencia para paquetes de 64 bits con retardo de 0–5 ciclos.

En esta configuración se obtuvo un promedio de 13511.4 ciclos, con un mínimo de 71 ciclos y un máximo de 26981 ciclos.

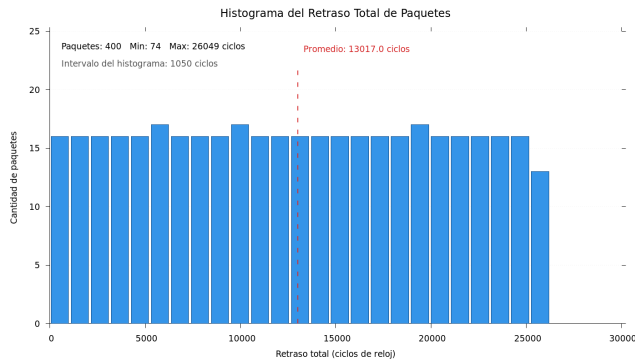


Fig. 15. Latencia para paquetes de 64 bits con retardo de 5–20 ciclos.

El promedio obtenido para esta configuración fue de 13017.0 ciclos, con un mínimo de 74 ciclos y un máximo de 26049 ciclos.

También se realizó una prueba adicional con paquetes de 64 bits y un intervalo de retardo de 20–100 ciclos.

Para esta prueba se obtuvo un promedio de 10649.2 ciclos, con un mínimo de 72 ciclos y un máximo de 21649 ciclos.

D. Resumen de las mediciones de latencia

La Tabla X resume los valores obtenidos en las diferentes configuraciones evaluadas.

En conjunto, los histogramas permiten observar la distribución de las latencias obtenidas durante las pruebas. Al aumentar el tamaño del paquete se observa una mayor cantidad de ciclos necesarios para completar una transferencia, lo cual es consistente con el carácter serial del bus. Las diferentes configuraciones de retardo también modifican la distribución de los tiempos registrados.

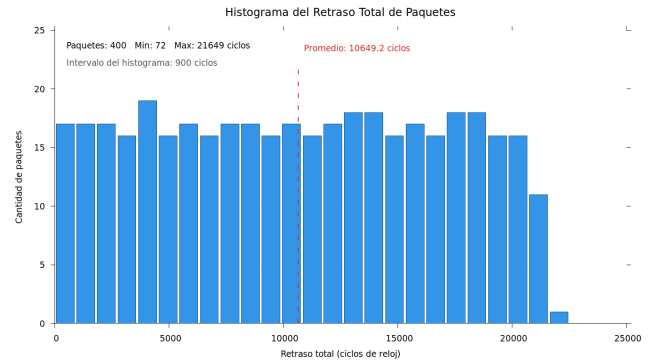


Fig. 16. Latencia para paquetes de 64 bits con retardo de 20–100 ciclos.

TABLE X
RESUMEN DE LAS MEDICIONES DE LATENCIA.

Tamaño	Retardo	Mínimo	Promedio	Máximo
16 bits	0–5	23	3887.4	7781
16 bits	5–20	26	3393.0	6849
16 bits	20–100	22	1043.0	2467
32 bits	0–5	39	7095.4	14181
32 bits	5–20	42	6601.0	13249
32 bits	20–100	40	4233.2	8849
64 bits	0–5	71	13511.4	26981
64 bits	5–20	74	13017.0	26049
64 bits	20–100	72	10649.2	21649

E. Conclusión de los resultados

Los resultados obtenidos muestran que las pruebas funcionales realizadas fueron completadas correctamente. Las transferencias punto a punto, el broadcast y el manejo de destinos inválidos fueron identificados y comprobados mediante la interacción entre el monitor, el scoreboard y el checker.

Además, las pruebas de latencia permitieron obtener mediciones para diferentes tamaños de paquete y condiciones de generación de tráfico. Los histogramas facilitan la visualización de estas mediciones y permiten comparar el comportamiento del bus bajo las distintas configuraciones utilizadas.